Chapter 2

A Historical Perspective

Ideas regarding the design of computational systems that are based on the notion of simulated evolution are not new. The purpose of this chapter is to provide a brief historical perspective. Readers interested in more details are encouraged to consult David Fogel's excellent survey of the field's early work (Fogel, 1998).

2.1 Early Algorithmic Views

Perhaps the place to start is the 1930s, with the influential ideas of Sewell Wright (Wright, 1932). He found it useful to visualize an evolutionary system as exploring a multi-peaked fitness landscape and dynamically forming clusters (demes) around peaks (niches) of high fitness. This perspective leads quite naturally to the notion of an evolutionary system as an optimization process, a somewhat controversial view that still draws both support and criticism. In any case, viewing evolution as optimization brought it in contact with many of the early efforts to develop computer algorithms to automate tedious manual optimization processes (see, for example, Box (1957)). The result is that, even today, optimization problems are the most dominate application area of evolutionary computation.

An alternate perspective is to view an evolutionary system as a complex, adaptive system that changes its makeup and its responses over time as it interacts with a dynamically changing landscape. This viewpoint leads quite naturally to the notion of evolution as a feedback-control mechanism responsible for maintaining some sort of system stasis in the face of change. Viewing evolution as an adaptive controller brought it in contact with many of the early efforts to develop computer algorithms to automate feedback-control processes. One of the earliest examples of this can be seen in Friedman (1956), in which basic evolutionary mechanisms are proposed as means of evolving control circuits for robots.

The tediousness of writing computer programs has inspired many attempts to automate this process, not the least of which were based on evolutionary processes. An early example of this can be seen in Friedberg (1959), which describes the design and implementation of a "Learning Machine" that evolved sets of machine language instructions over time. This theme has continued to capture people's imagination and interest, and has resulted

in a number of aptly named subareas of evolutionary computation such as "evolutionary programming" and "genetic programming".

2.2 The Catalytic 1960s

Although the basic idea of viewing evolution as a computational process had been conceived in various forms during the first half of the twentieth century, these ideas really took root and began to grow in the 1960s. The catalyst for this was the increasing availability of inexpensive digital computers for use as a modeling and simulation tool by the scientific community. Several groups became captivated by the idea that even simple evolutionary models could be expressed in computational forms that could be used for complex computer-based problem solving. Of these, there were three groups in particular whose activities during the 1960s served to define and shape this emerging field.

At the Technical University in Berlin, Rechenberg and Schwefel began formulating ideas about how evolutionary processes could be used to solve difficult real-valued parameter optimization problems (Rechenberg, 1965). From these early ideas emerged a family of algorithms called "evolution strategies" which today represent some of the most powerful evolutionary algorithms for function optimization.

At UCLA, during the same time period, Fogel saw the potential of achieving the goals of artificial intelligence via evolutionary techniques (Fogel et al., 1966). These ideas were explored initially in a context in which intelligent agents were represented as finite state machines, and an evolutionary framework called "evolutionary programming" was developed which was quite effective in evolving better finite state machines (agents) over time. This framework continues to be refined and expanded today and has been applied to a wide variety of problems well beyond the evolution of finite state machines.

At the University of Michigan, Holland saw evolutionary processes as a key element in the design and implementation of robust adaptive systems that were capable of dealing with an uncertain and changing environment (Holland, 1962, 1967). His view emphasized the need for systems which self-adapt over time as a function of feedback obtained from interacting with the environment in which they operate. This led to an initial family of "reproductive plans" which formed the basis for what we call "simple genetic algorithms" today.

In each case, these evolutionary algorithms represented highly ideal models of evolutionary processes embedded in the larger context of a problem-solving paradigm. The spirit was one of drawing inspiration from nature, rather than faithful (or even plausible) modeling of biological processes. The key issue was identifying and capturing computationally useful aspects of evolutionary processes. As a consequence, even today most evolutionary algorithms utilize what appear to be rather simplistic assumptions such as fixed-size populations of unisex individuals, random mating, offspring which are instantaneously adult, static fitness landscapes, and so on.

However, a formal analysis of the behavior of even these simple EAs was surprisingly difficult. In many cases, in order to make such analyses mathematically tractable, additional simplifying assumptions were necessary, such as assuming an infinite population model and studying its behavior on simple classes of fitness landscapes. Unfortunately, the behavior

of realizable algorithms involving finite populations observed for finite periods of time on complex fitness landscapes could (and did) deviate considerably from theoretical predictions. As a consequence, the field moved into the 1970s with a tantalizing, but fuzzy, picture of the potential of these algorithms as problem solvers.

2.3 The Explorative 1970s

The initial specification and analysis of these simple evolutionary algorithms (EAs) in the 1960s left two major issues unresolved: 1) characterizing the behavior of implementable systems, and 2) understanding better how they might be used to solve problems. To implement these simple EAs, very specific design decisions needed to be made concerning population management issues and the mechanisms for selecting parents and producing offspring. It was clear very early that such choices could significantly alter observed EA behavior and its performance as a problem solver. As a consequence, much of the EA research in the 1970s was an attempt to gain additional insight into these issues via empirical studies and extensions to existing theory. Most of this activity occurred within the three groups mentioned earlier and resulted in the emergence of three distinct species of EAs: evolutionary programming, evolution strategies, and genetic algorithms.

2.3.1 Evolutionary Programming

The evolutionary programming (EP) paradigm concentrated on models involving a fixed-size population of N parents, each of which produced a single offspring. The next generation of N parents was determined by combining both parents and children into a single population of size 2N, rank ordering them by fitness and allowing only the top N to survive. Initial work (Fogel et al., 1966) had proposed both asexual reproduction (single parent with a mutation operator) and sexual reproduction (two parents combining to form offspring via a recombination operator). Since the individuals being evolved were finite state machines (FSMs), mutation took the form of adding/deleting states/arcs, and recombination required combining pieces of two FSMs into a new FSM.

Empirical studies focused on a variety of issues, including appropriate initialization strategies (initial population of FSMs), appropriate frequencies for the various forms of mutation (arcs, states), and an appropriate recombination operator. Extensive simulation studies noted the difficulty of defining an effective recombination operator in this context, but reported impressive results in evolving useful FSMs with only asexual reproduction and mutation. The size of the population and the intensity of mutation varied from application to application.

2.3.2 Evolution Strategies

The focus of the evolution strategy (ES) paradigm was on real-valued function optimization. Hence, individuals were naturally represented as vectors of real numbers. Initial work involved $(1 + \lambda)$-ES models in which 1 parent produced λ offspring and the fittest of the $1 + \lambda$ individuals was selected to be the *single* parent for the next generation of offspring. In this context asexual reproduction took the form of mutating one or more of the parent's

gene values (real numbers) via a normally distributed perturbation $G(0, \sigma)$ with zero mean and a standard deviation of σ.

Empirical studies indicated that the performance of these ESs on particular functions to be optimized was highly sensitive to the choice of $G(0, \sigma)$. This resulted in a very early commitment to the design of an adaptive mutation operator. The form this took extended the representation of individuals from a vector of size N (N parameter values) to a vector of size 2N, in which the additional N genes represented the variance σ_i to be used to mutate parameter i using a normally distributed perturbation $G(0, \sigma_i)$. As a consequence, rather than being predetermined, mutation operators coevolved over time in response to the characteristics of the function being optimized.

What remained was to specify which (meta-) operators would be used to mutate the σ_i. This resulted in the now-famous 1:5 rule which is derived from a more formal analysis of optimal rates of convergence. Informally, this rule states that if more than one of five mutations using $G(0, \sigma_i)$ is successful (in the sense that an offspring has higher fitness than its parent), then mutation is in danger of being too exploitative (too small steps), and σ_i needs to be increased. If the rate of success falls below 20%, mutation is viewed as too explorative (too large steps) and σ_i is decreased.

With this adaptive mutation operator in place and an appropriately chosen λ, impressive results were reported for a wide range of difficult functions when using comparative results from more traditional optimization techniques (see for example, Schwefel (1975)).

2.3.3 Genetic Algorithms

In contrast to the early EP and ES work, the early genetic algorithm (GA) focus was on developing more application-independent algorithms. The approach taken used a genetic-like "universal" string representation for individuals along with string-oriented reproductive operators for producing offspring. The easiest string representations to analyze and visualize were fixed-length binary strings. In this context, mutation took the form of a "bit flip" with a fixed probability, and recombination took the form of a 1-point crossover operator, in which a randomly selected initial sequence of genes from one parent is combined with the remaining sequence of genes from a second parent to produce offspring with features of both parents.

Most of the early studies involved "generational" GAs, in which a fixed-size population of N parents produce a new population of N offspring, which unconditionally replaces the parent population regardless of fitness. However, parents are stochastically selected to produce offspring in proportion to their fitness. Thus, the fitter parents contribute significantly more "genetic material" to the next generation.

In order to apply these GAs to a particular problem, the individuals to be evolved over time needed to be represented as fixed-length binary strings. This was an advantage in the sense that few, if any, changes to the GA were required when attacking a new problem domain. However, it quickly became clear that the effectiveness of a GA depended on the choice of the phenotype-to-genotype mapping (and its inverse).

GAs were studied intensively from both an analytical and empirical point of view. The fixed-length string representation led naturally to a series of "schema theorems" (Holland, 1975) concerning the behavior of simple GAs. Much of the early empirical GA studies

involved problems in which the fitness landscape was defined by carefully chosen time-invariant memoryless functions whose surfaces were well understood and involved simple binary string representations (see, for example, De Jong (1975)). Out of these early studies emerged a picture of a GA as a robust adaptive search procedure, which was surprisingly effective as a global search heuristic in spite of its universal properties.

2.4 The Exploitative 1980s

What emerged out of the 1970s was a collection of canonical evolutionary algorithms which had evolved to the point of exhibiting rather impressive performance when applied to a focused set of problems. The next obvious step was to use this core set of algorithms as the basis for scaling up to more complex problems and for developing new EA-based problem solvers for other problem domains. The activities during the 1980s resulted in significant developments on both fronts. A brief summary follows.

2.4.1 Optimization Applications

Perhaps the most natural application of EAs is as an optimizer, and so considerable effort was put into applying EAs to various kinds of optimization problems. The simplest of these are parameter optimization problems in which a fixed set of parameters with fixed sets of permissible parameter values define an N-dimensional space over which a performance function is defined. As existing EAs were applied to a broader range of such problems, the results varied from excellent to poor.

In the case of Evolution Strategies (ESs), there were a number of difficulties in scaling up to high-dimensional problems, handling interactions among parameters, and dealing with highly multimodal surfaces. This led to several important extensions to the $(1 + \lambda)$-ES model. The first was a generalization to a multi-parent $(\mu + \lambda)$ model in which the top μ of the $\mu + \lambda$ individuals were selected as the parent population for the next generation. A second variation involved the use of a "generational" model (μ, λ) in which the top μ individuals from the current set of λ offspring are selected as the parent population for the next generation (and no parents survive). In order to handle parameter interactions more effectively, estimates of the N^2 pairwise correlations could be encoded as well and used to guide pairwise mutations of parameter values.

Similarly, as Genetic Algorithms (GAs) were applied to a broader range of parameter optimization problems, a number of difficulties arose. It was observed that GAs frequently did not exhibit a "killer instinct" in the sense that, although they rapidly locate the region in which a global optimum exists, they did not converge to the optimum with similar speed. This turned out to be a property of the fitness-proportional selection mechanism used in simple GAs. A number of solutions emerged, including dynamically scaling the fitness function in order to maintain sufficient selective pressure, or switching to an alternative selection mechanism such as rank or tournament selection (Goldberg, 1989; Whitley, 1989a).

It was also observed that GAs could be "deceived" into converging to non-optimal points in the search space. This led to theoretical analyses of deceptiveness and also practical insights into how such situations might be detected and/or overcome (Goldberg, 1989).

Additional GA performance improvements were frequently obtained by singling out the best and/or worst individuals in the current population for special treatment (elitist policies) (De Jong, 1975; Whitley, 1989a). The effect is to shift the balance toward more exploitation and less exploration which, for some classes of functions, works quite well. However, if no *a priori* information about the functions to be optimized is available, this can result in suboptimal hill-climbing behavior on multi-peaked functions.

The internal representation used by a GA can also have an effect of optimization performance, in that points which are close in parameter space may not be close when mapped onto the standard internal binary representation. Such observations led to experiments with alternative binary representations, such as Gray codes, in an attempt to avoid these problems. Other researchers abandoned the internal string representation entirely and switched to real-valued vector representations.

Additional performance improvements were obtained by building specialized EAs which took advantage of *a priori* knowledge about a particular problem class. This usually took the form of specialized representations and specialized operators for important subclasses such as continuous real-valued functions, combinatorial optimization problems (e.g., traveling salesperson problems), and constrained optimization problems (job shop scheduling problems, etc.).

2.4.2 Other EA Applications

Although function optimization was a dominant GA application area, there was also considerable effort spent in applying GAs to other difficult problem areas. Classifier systems (Holland, 1986; Booker, 1982) were developed as simple models of cognition with GAs as the key component for adaptation and learning. Attempts were made to evolve more complex objects such as neural networks (Harp et al., 1989; de Garis, 1990), collections of rules for executing tasks (Smith, 1983; Grefenstette, 1989), and Lisp code (Koza, 1989). In each case, there were difficult and challenging issues raised regarding how best to harness the power of these simple GAs for the intended purpose.

One clearly emerging theme was the difficulty of representing complex, nonlinear objects of varying size using the traditional fixed-length string representation. A second theme which emerged was the need to understand better how GAs behave in environments with changing and/or noisy feedback. The increased availability of parallel machines gave rise to a third important theme: how to exploit the inherent parallelism of GAs.

2.4.3 Summary

By the end of the 1980s several important trends were in evidence. First, the focus on effective problem-solving performance resulted in new and more powerful EAs, but this was accomplished in most cases at the expense of decreasing their plausibility as models of biological evolutionary systems. These extensions to the canonical EAs resulted in a new, more complex class of algorithms that were well beyond the initial theoretical analyses, creating a need to revisit and extend the theoretical models.

In addition, the level of activity of the field had increased to the point that it required its own conferences, starting with the GA community (ICGA-85, ICGA-87 and ICGA-89),

and with plans underway in the ES community for PPSN-90, plans in the EP community for EP-90, and the theoretical community planning FOGA-90.

2.5 The Unifying 1990s

Up to this point in time, much of the research and development of the primary EA paradigms (EP, ES, and GA) was done independently without much interaction among the various groups. The emergence of various EA conferences in the late 1980s and early 1990s, however, changed all that as representatives from various EA groups met, presented their particular viewpoints, challenged other approaches, and were challenged in return.

The result of these first interactions was a better understanding of the similarities and differences of the various paradigms, a broadening of the perspectives of the various viewpoints, and a feeling that, in order to continue to develop, the field as a whole needed to adopt a unified view of these evolutionary problem solvers. The immediate effect was an agreement on the term "evolutionary computation" (EC) as the name of the field and a commitment to start the field's first journal, *Evolutionary Computation*.

A second effect of these interactions was a noticeable crossbreeding of ideas. The ES community found that sexual reproduction with various forms of recombination improved the performance of their function optimizers (Bäck and Schwefel, 1993). The GA community found that problem-specific representations and operators frequently improved the performance of GA-based problem solvers (Michalewicz, 1994). The EP community extended their domain of application to other areas such as function optimization, incorporating ES-like representations and operators. What emerged from all this was a dizzying array of new EAs such as Genitor, Genetic Programming (GP), messy GAs, Samuel, CHC, Genocoop, etc., none of which fit nicely into the canonical EA paradigms and whose capabilities challenged old tenets. It became increasingly clear that a broader and more fundamental view of the field was required (De Jong and Spears, 1993).

A third effect of these interactions was a general sense of the need to revisit many of the fundamental assumptions and underlying theory in order to strengthen and generalize the basic EA paradigms. The FOGA workshops, started in 1990, have resulted in a rebirth of activity in EA theory with new characterizations and analyses in terms of formae (Radcliffe, 1991), Markov models (Vose, 1992), PAC analysis (Ros, 1992), breeder theory (Mühlenbein and Schlierkamp-Voosén, 1993), order statistics (Bäck, 1994), and so on.

2.6 The Twenty-first Century: Mature Expansion

Although really only forty years old, by the end of the 1990s the field of evolutionary computation had the look and feel of a mature scientific discipline. The community now supported three archival journals:

- *Evolutionary Computation*
- *IEEE Transactions on Evolutionary Computation*
- *Genetic Programming and Evolvable Machines*

The amount of interest in evolutionary computation research and applications was reflected by a wide range of peer-reviewed conferences including:

- GECCO: Genetic and Evolutionary Computation Conference
- CEC: Congress on Evolutionary Computation
- PPSN: Parallel Problem Solving from Nature
- FOGA: Foundations of Genetic Algorithms

In addition, the field was producing a steadily increasing number of excellent books including:

- Goldberg (1989), Davis (1991), Holland (1992), Koza (1992)
- Michalewicz (1994), Kinnear (1994), Koza (1994), Rechenberg (1994)
- Fogel (1995), Schwefel (1995), Bäck (1996)
- Banzhaf et al. (1998), Fogel (1998), Mitchell (1998)
- Bentley (1999), Koza et al. (1999), Vose (1999)
- Spears (2000), Bentley and Corne (2001), Deb (2001)
- Langdon and Poli (2001),Goldberg (2002), Coello et al. (2002)

At the same time there was no sense of complacency among the members of the evolutionary computation community. Although they had a reasonably good understanding of simple evolutionary algorithms and their applications, they were continually being confronted with new and more complex problem areas that tested the limits of their current systems. This has led various groups to focus on a number of fundamental extensions to existing EA models that appear to be necessary in order to improve and extend their problem solving capabilities. Among the issues being explored are:

- pros and cons of phenotype/genotype representations
- the inclusion of Lamarckian properties
- nonrandom mating and speciation
- decentralized, highly parallel models
- self-adapting systems
- coevolutionary systems
- agent-oriented models

2.7 Summary

Though a relatively new field, evolutionary computation's roots are quite rich and varied, and can be seen as far back as the 1930s. This historical perspective provides the context that has shaped the field as we know it today, and provides the basis for the unified view presented in the following chapters.

Ch

Ca
Al

3.1

A very
ideas a
its feat
three h
strateg
comput

3.2

Recall
which v
objecti
domly
simulat
at rand
variatic
genome
rent va
surviva
the obj
the old
Thi
immedi
alterna
a fixed